

COUPLING

IN SOFTWARE DESIGN



Moisés Montes Iglesias

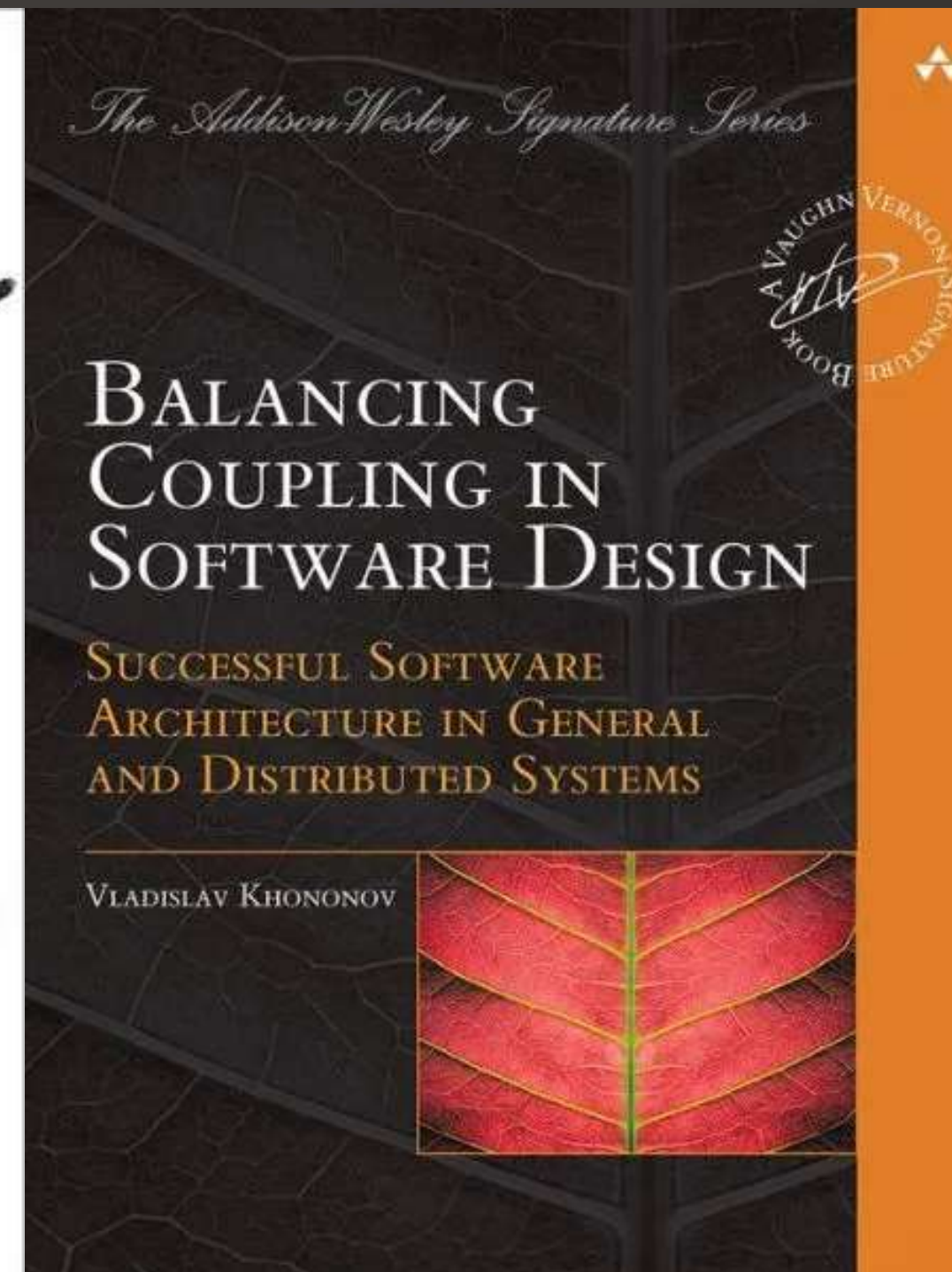
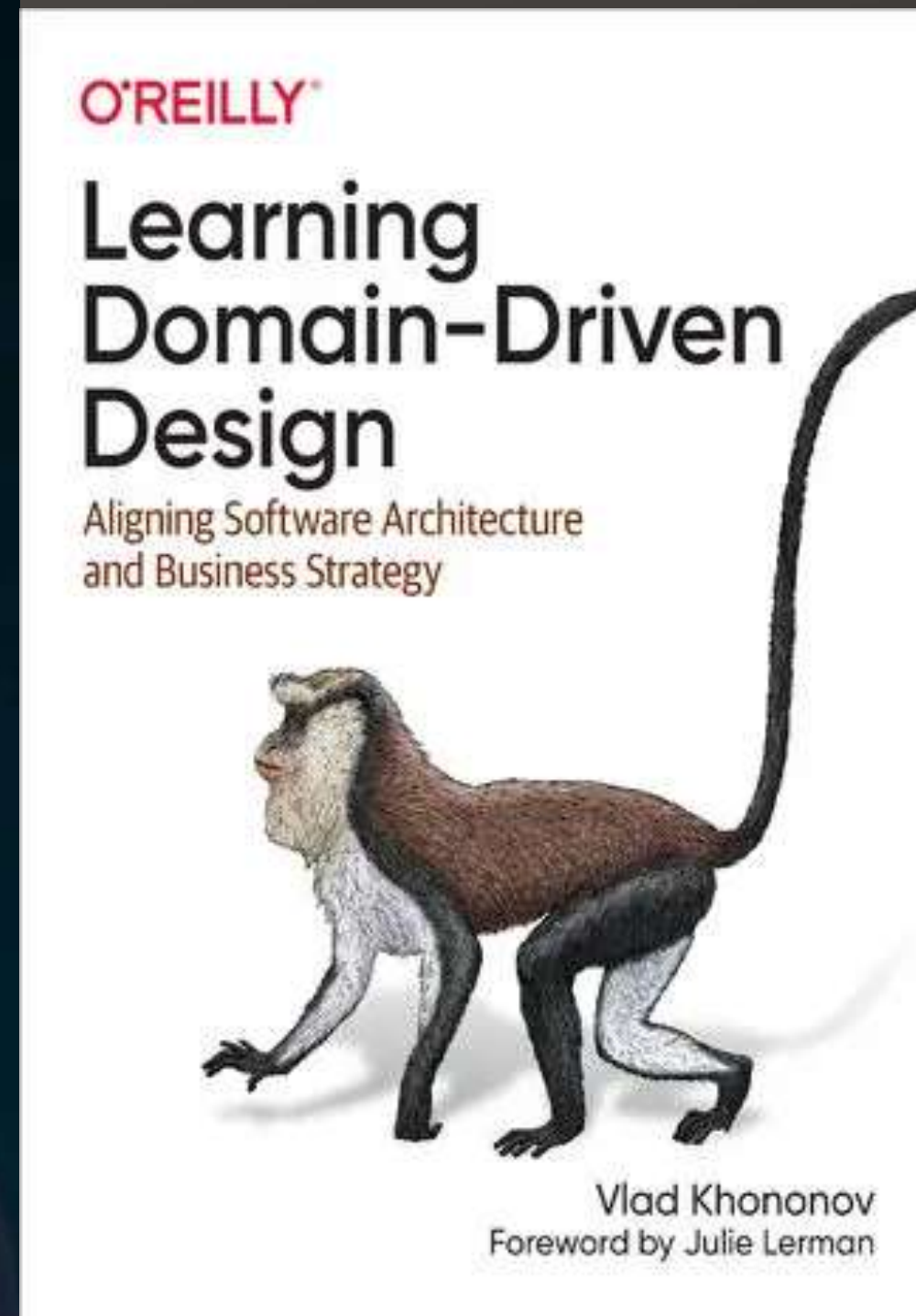
Uo296102

Pedro Díaz González

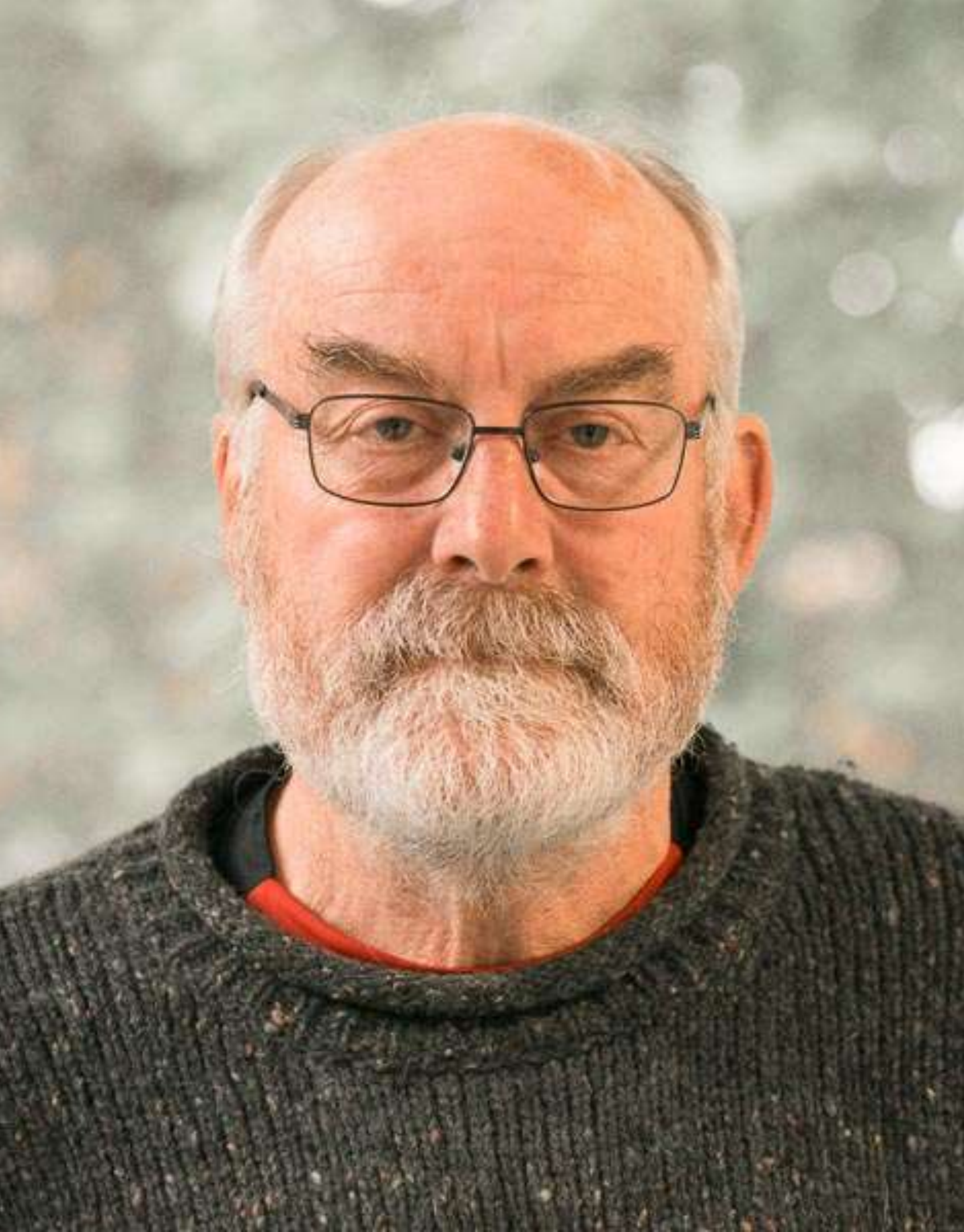
uo294790

¿Quién es Vlad Khononov?

Learning Domain-Driven Design Balancing Coupling in Software Design

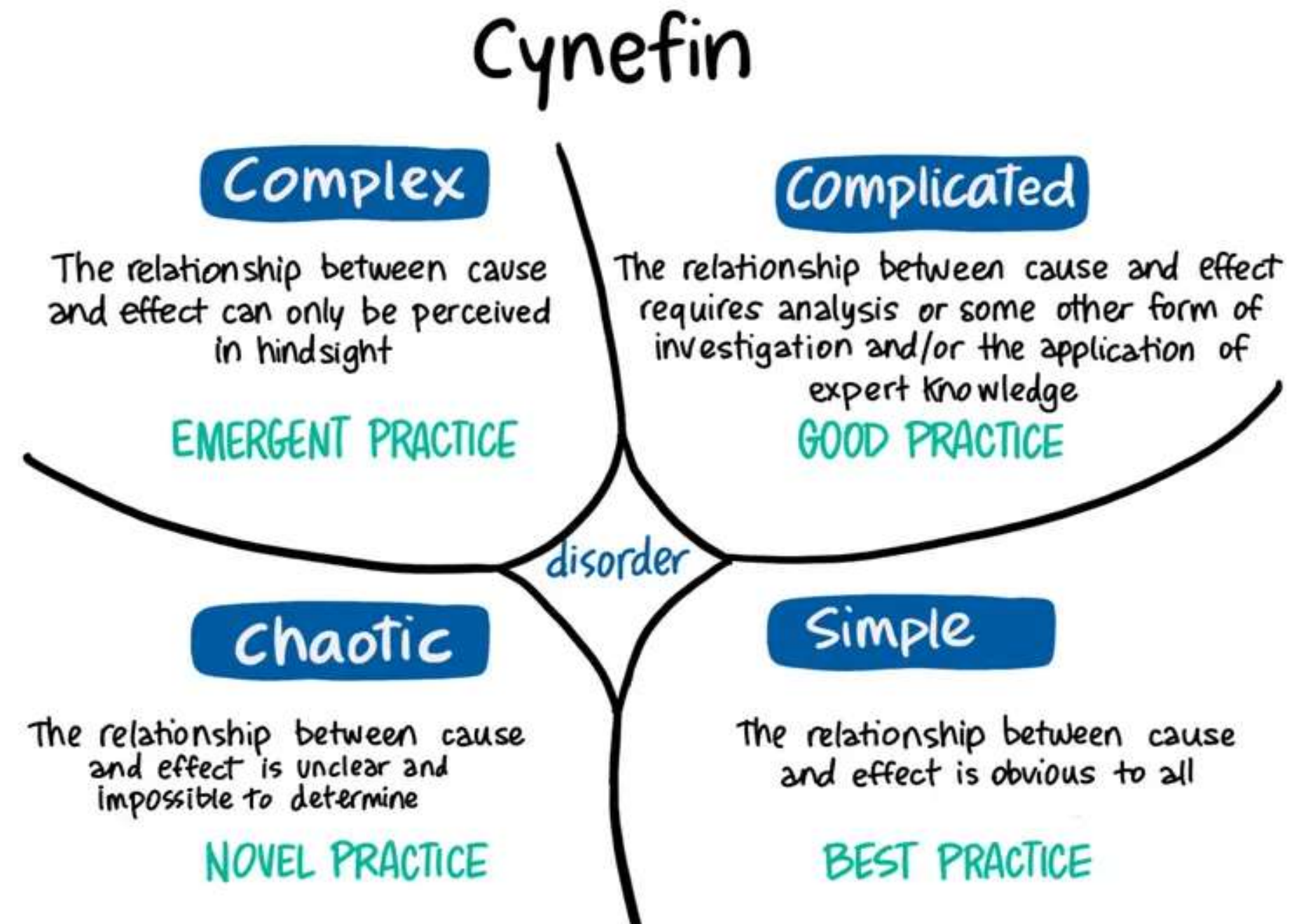


¿Tienes una perspectiva diferente sobre cómo se relacionan entre sí la complejidad y el acoplamiento, o quizá sobre las definiciones de esos términos?

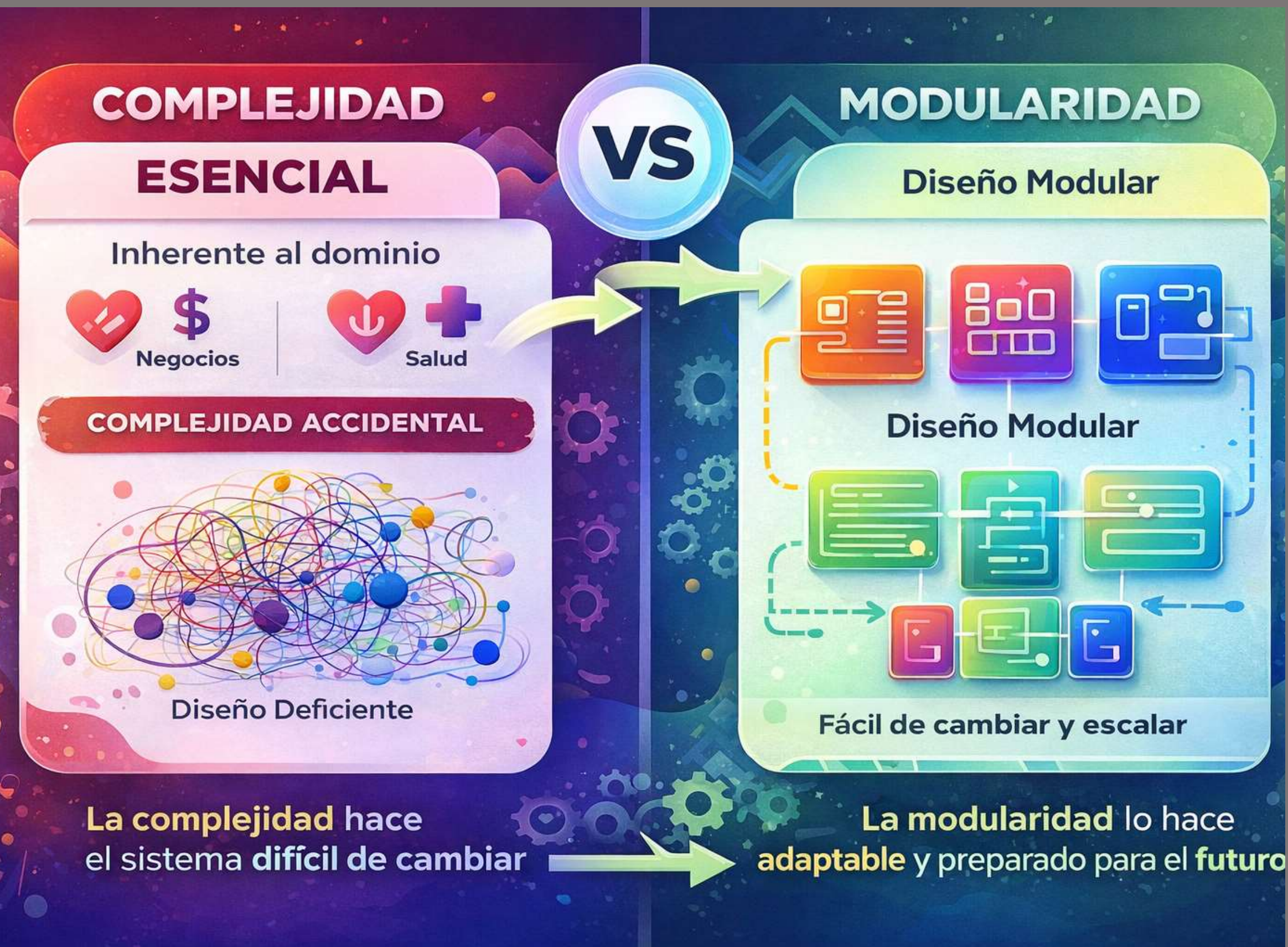


¿Qué opina Vlad sobre como se relacionan la complejidad y el acoplamiento?

- El marco Cynefin de Dave Snowden
- *"La complejidad es una función de acción y el resultado de dicha acción"*
- ¿Cuál es el marco de Vlad?



Complejidad vs Modularidad



Tipos de complejidad

- Complejidad del dominio
- Complejidad del diseño

¿Por qué buscamos la modularidad, que beneficios nos da?

Por lo tanto, según Vlad, ¿qué es un módulo?

¿Qué es un Módulo?

— 50 años después de David Parnas... —

¿Qué es la modularidad?

— Saber Dónde Cambiar —

Identificar exactamente qué parte modificar

Aquí está

Modularidad: —

Dónde Cambiar y **Cuáles** son los **Efectos** del Cambio

— Sin Sorpresas —

¿Qué diseño de módulo queremos?

— Efectos Predecibles —

Confianza en el Resultado



— Sin Sorpresas —
¡Todo Bajo Control!

- ¿Qué es la modularidad?
- ¿Qué diseño de módulo queremos conseguir?

¿Mediante qué tres criterios se definía antes un módulo?

- Funcionalidad bien definida
 - Interfaz pública
 - Compilado de forma independiente
- Ejemplos actuales

- Namespace de C#
- Clases y objetos ¿Debería?
- Método de una clase ¿Debería?

¿Qué podemos decir entonces de los módulos?

Los módulos son fractales y
presentan niveles de abstracción

“El propósito de una abstracción es crear un
nuevo nivel semántico en el que se pueda ser
absolutamente preciso”



Resumen y conclusión de lo que hemos visto hasta ahora

Vlad Khononov

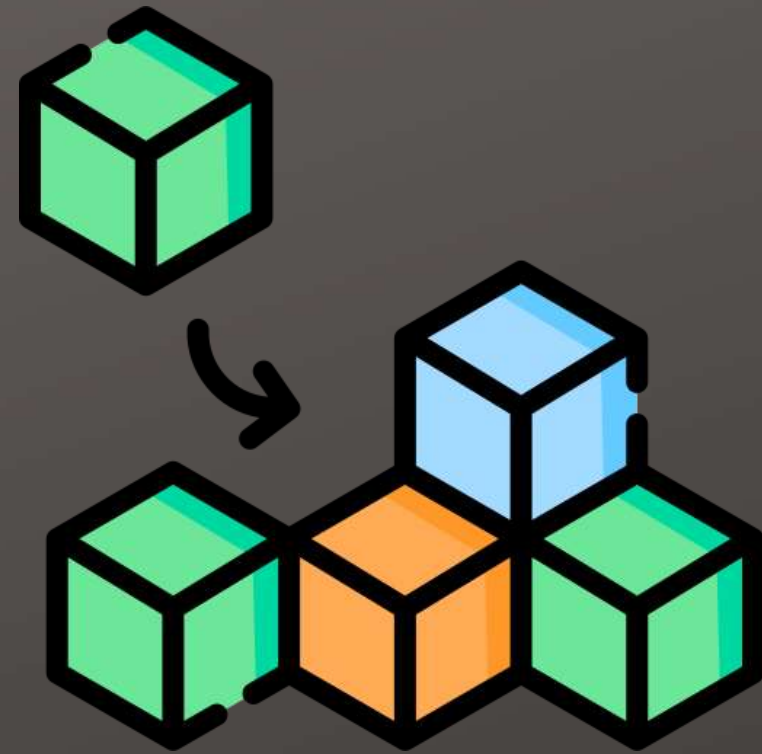
¿Qué es la complejidad?

¿Qué es un módulo?

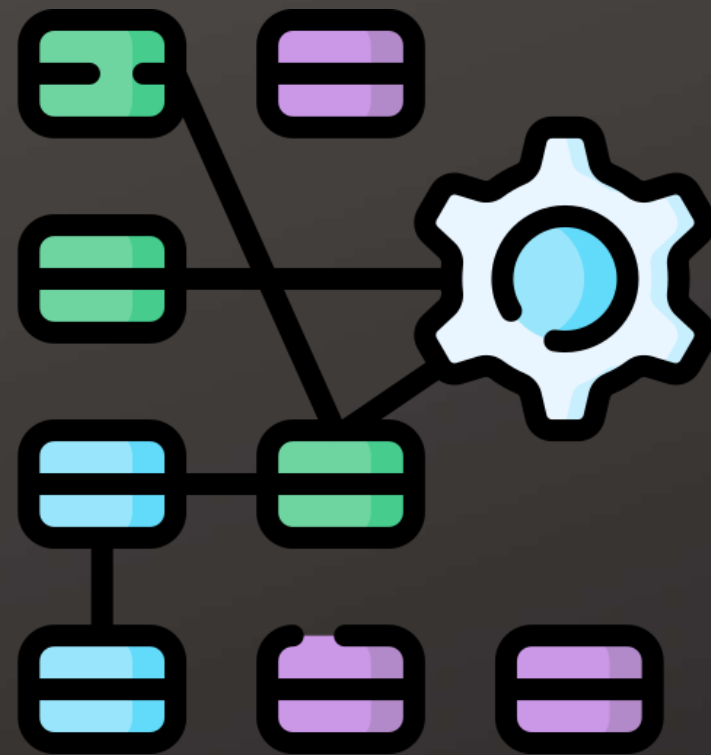


“Cada una de las capas o niveles de abstracción gestionan la complejidad.”

“Los módulos se pueden encontrar en diferentes niveles de abstracción. Todos ellos deben gestionar la complejidad en una jerarquía de tipo fractal”



¿Es la modularidad la única forma de lidiar con la complejidad?



- **Sí: siempre hay modularidad y complejidad.**
- **Elige entre avanzar modular o avanzar rápido.**
- **Una interfaz simple puede ocultar complejidad interna.**
- **No busques perfección: empieza y mejora con el tiempo.**
- **A veces hay que aceptar atajos.**

Historia del Acoplamiento

El acoplamiento ha cambiado con el diseño de software.

- **Años 70: Acoplamiento de Módulos**

Se definía con 6 niveles y lenguajes de la época.

- **Actualmente: Fuerza de Integración**

Khononov propone 4 niveles más simples y claros.

Fuerza de Integración: Los 4 Niveles de Vlad Khononov

Acoplamiento por contrato:

Solo interfaces y contratos.

Acoplamiento de modelo:

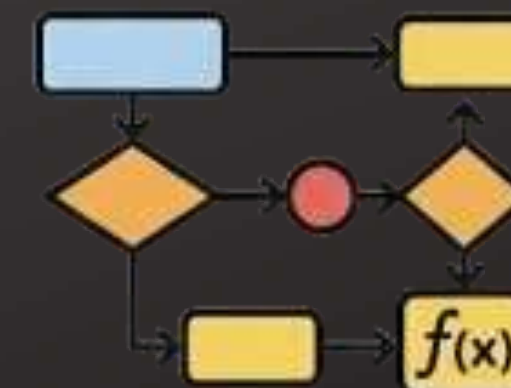
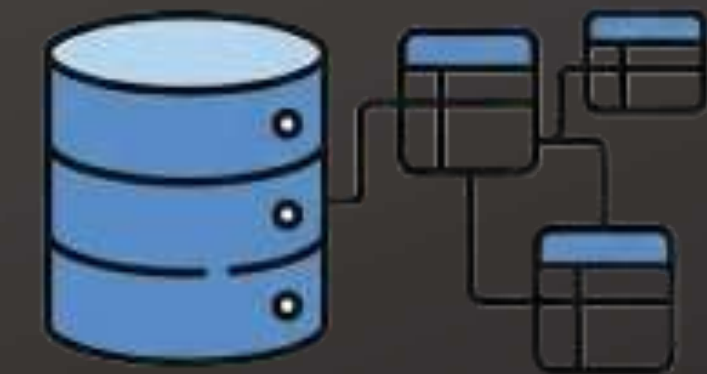
Comparte datos o modelo.

Acoplamiento funcional:

Comparte lógica de negocio.

Acoplamiento intrusivo:

Depende de detalles internos.



Connascence: Otro Modelo de Análisis

Surgimiento

- Modelo de los 90
- Enfocado en OO

Más detalle

- Describe dependencias específicas
- Entre componentes

Diferencia clave

- Acoplamiento: qué se comparte
- Connascence: cómo se comparte

Enfoque

- Fuerza de integración para clasificar
- Connascence para medir intensidad

Aspectos del Acoplamiento:

Conocimiento

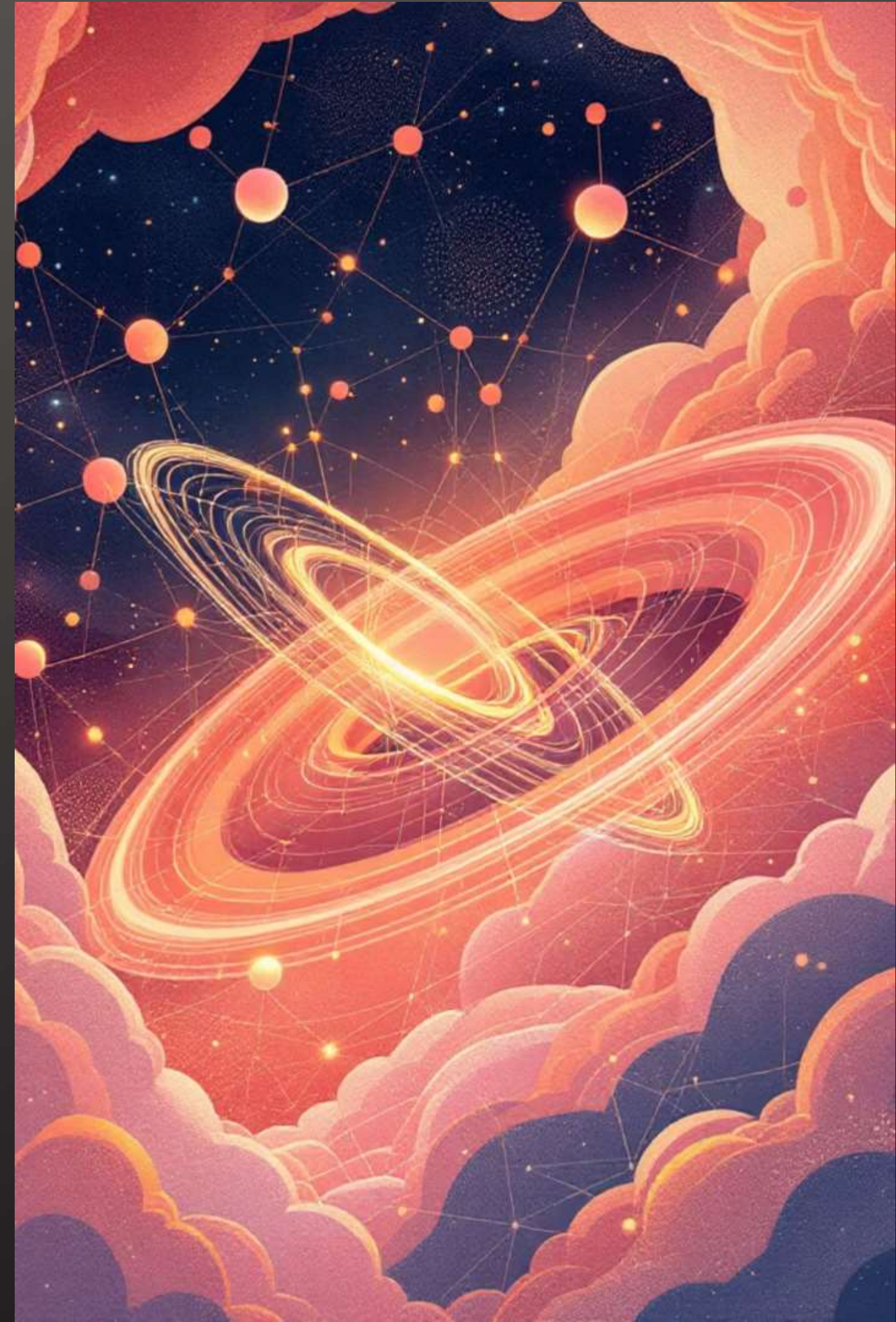
- Los módulos deben conocerse a través de una interfaz pública.
- Más información compartida = más cambios en cascada.
- Reduce el conocimiento al mínimo necesario.

Espacio

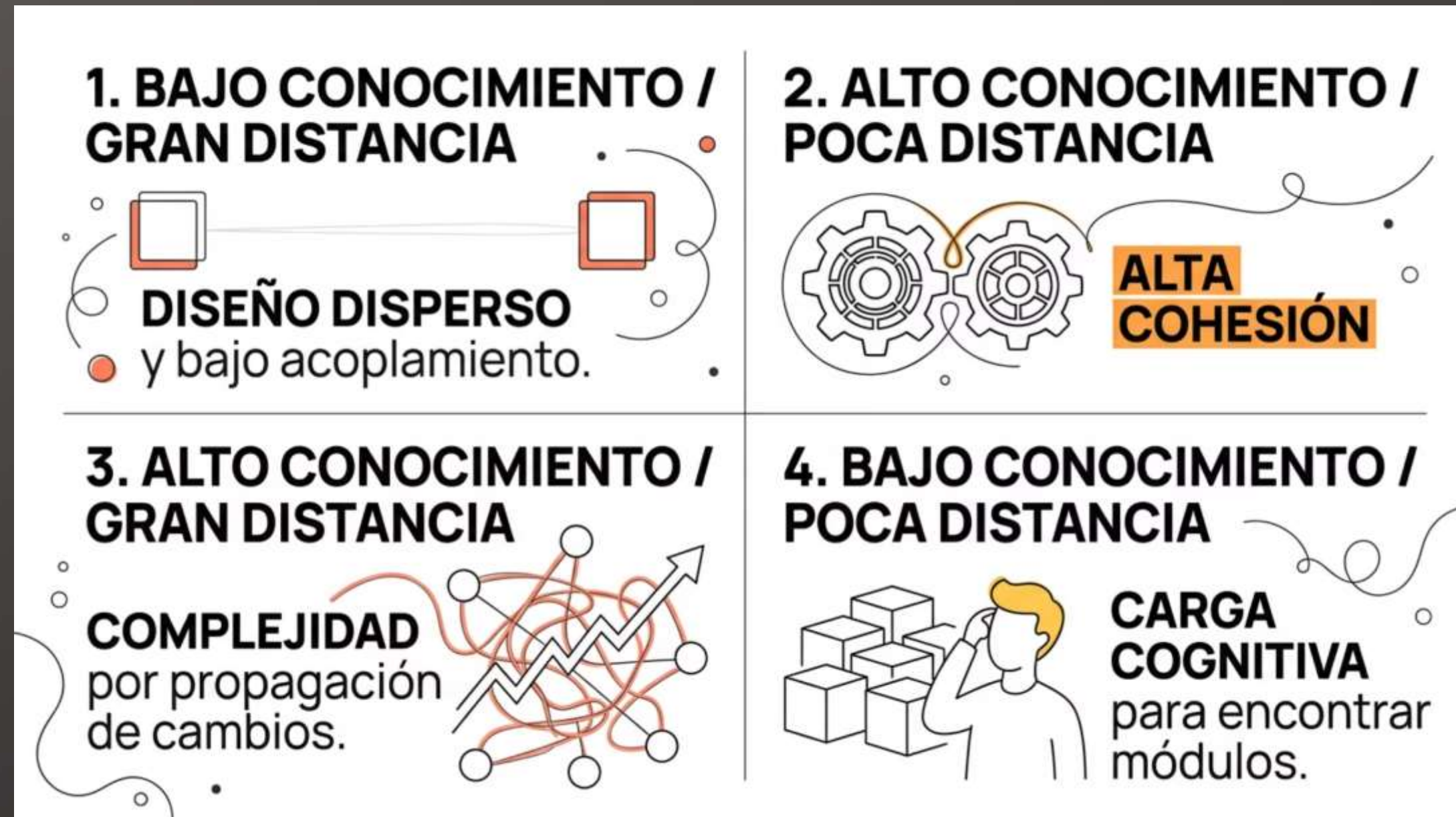
- Mayor distancia entre módulos = mayor impacto ante cambios.
- Mis archivo/carpeta = menor efecto cascada.
- A menor distancia, más fácil coordinar cambios.

Idea clave: buen diseño = acoplamiento controlado.

Estrategia: comparte lo mínimo; acerca lo que no puedas limitar.



Combinaciones de Conocimiento y Espacio



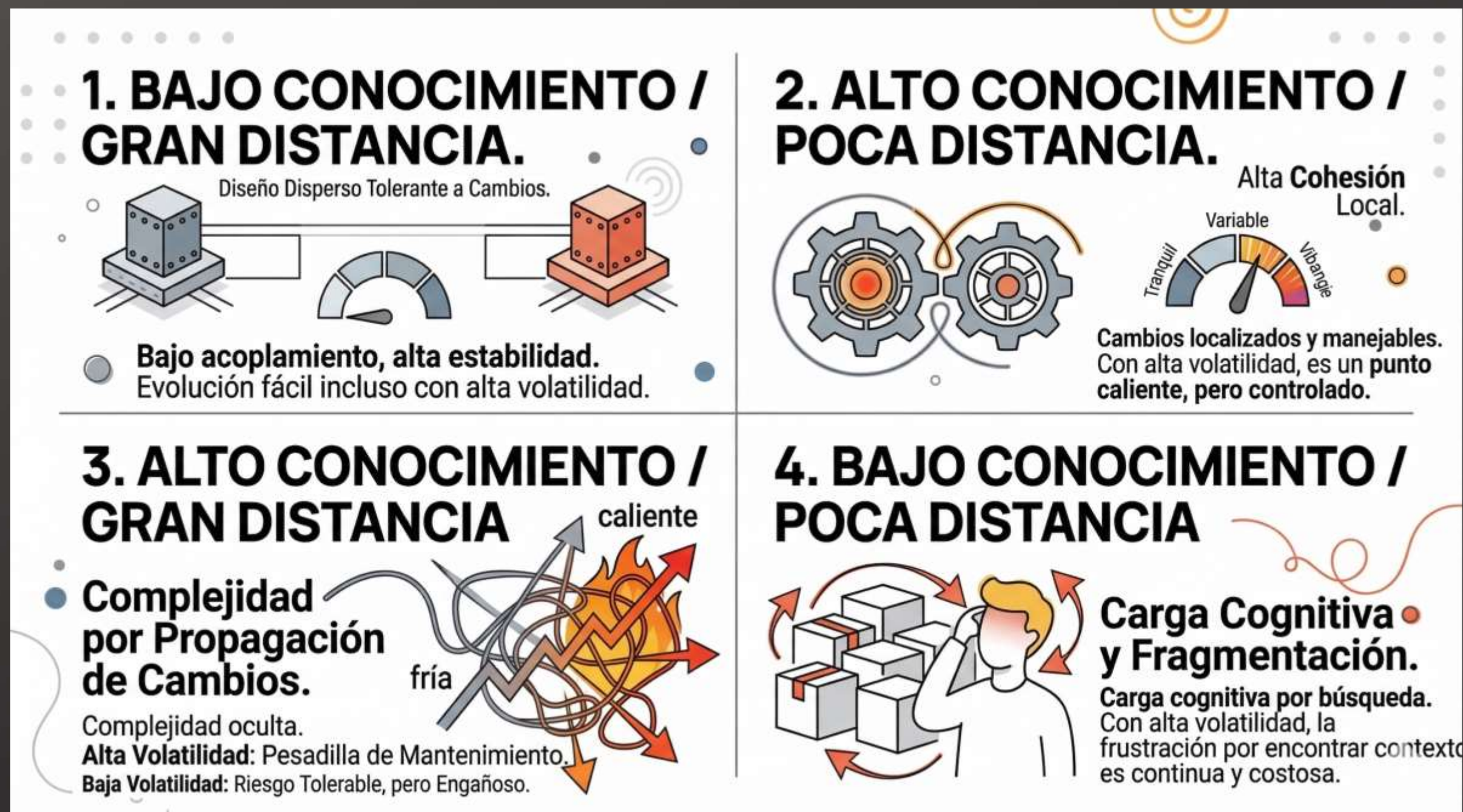
- Valores opuestos (uno alto y otro bajo) → modularidad.
- Valores iguales (ambos altos o ambos bajos) → complejidad.

La Tercera Dimensión: Volatilidad

Volatilidad: frecuencia con la que cambian los componentes.

La distancia entre módulos importa más cuando los cambios son frecuentes.

- **Baja volatilidad:** más complejidad tolerable.
- **Alta volatilidad:** diseño más cuidadoso.



Conclusión:

Las tres dimensiones clave del diseño:

- **Fuerza de integracion**
- **Distancia**
- **Volatilidad**

En conjunto:

- **Distinta → modularidad.**
- **Igual → complejidad.**
- **Baja volatilidad: más tolerancia.**
- **Alta volatilidad: más equilibrio.**

**El acoplamiento está en toda decisión de diseño.
Entenderlo es clave para crear software mantenible.**





Muchas gracias
por su atencion!!!

¿Alguna pregunta?